

Software Engineering for CS

Week 2

INSTRUCTOR : DR. NATASH ALI MIAN

School of Computer and Information Technology

Beaconhouse National University

Software Lifecycle Models

The way you organize your activities

Lifecycle model is a series of steps through which the product progresses

Process Models

A process model is a simplified description of a process presented from a particular perspective

There may be several different models of the same process

No single model gives a complete understanding of the process being modeled

Variations in Process Models

A process model is produced on the anticipated need for that model. We may need

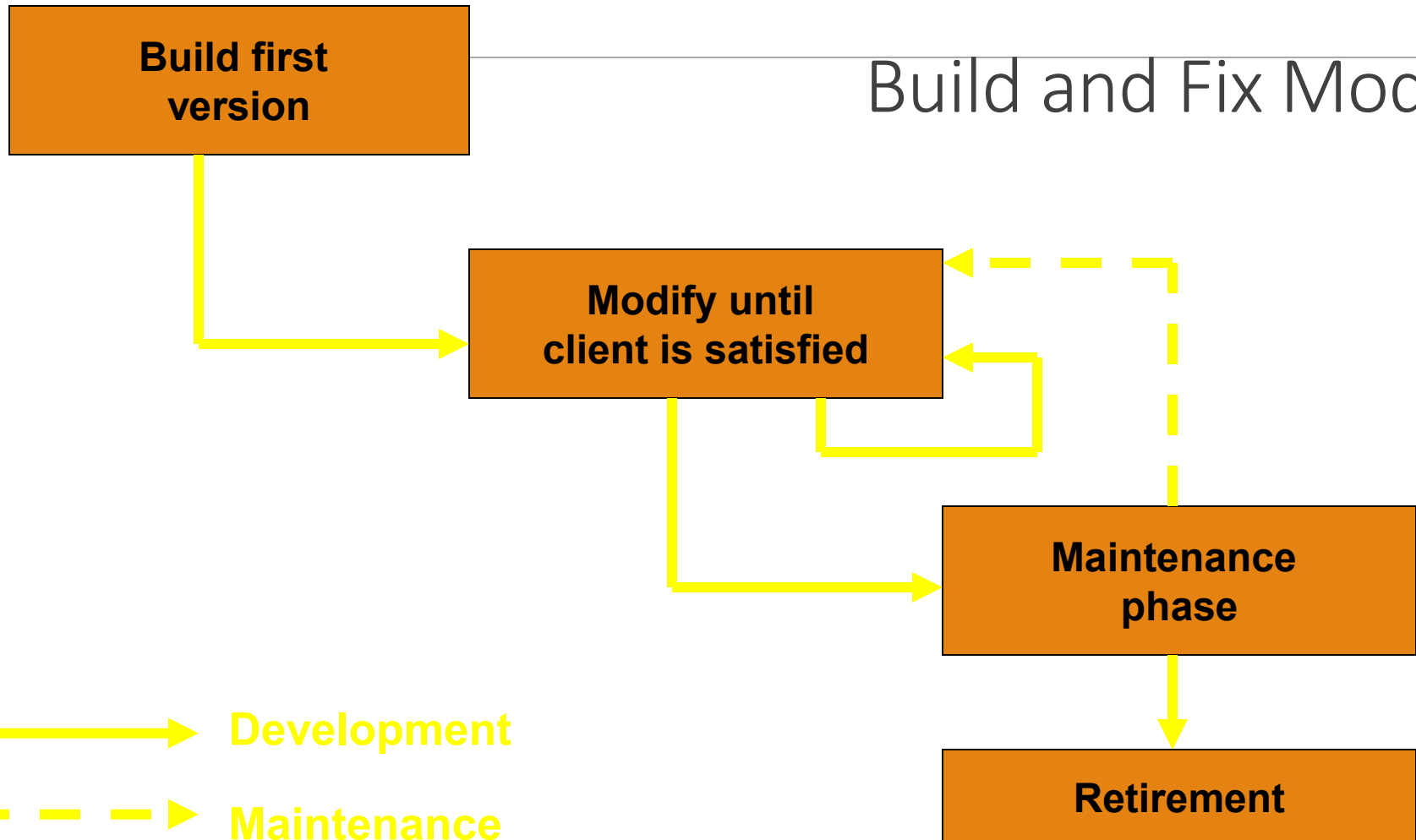
- A model to help explain how process information has been organized
- A model to help understand and improve a process
- A model to satisfy some quality management standard

Software Life-Cycle Steps

Life-cycle model (formerly known as process model)

- Requirements phase
- Specification phase
- Design phase
- Implementation phase
- Integration phase
- Maintenance phase
- Retirement

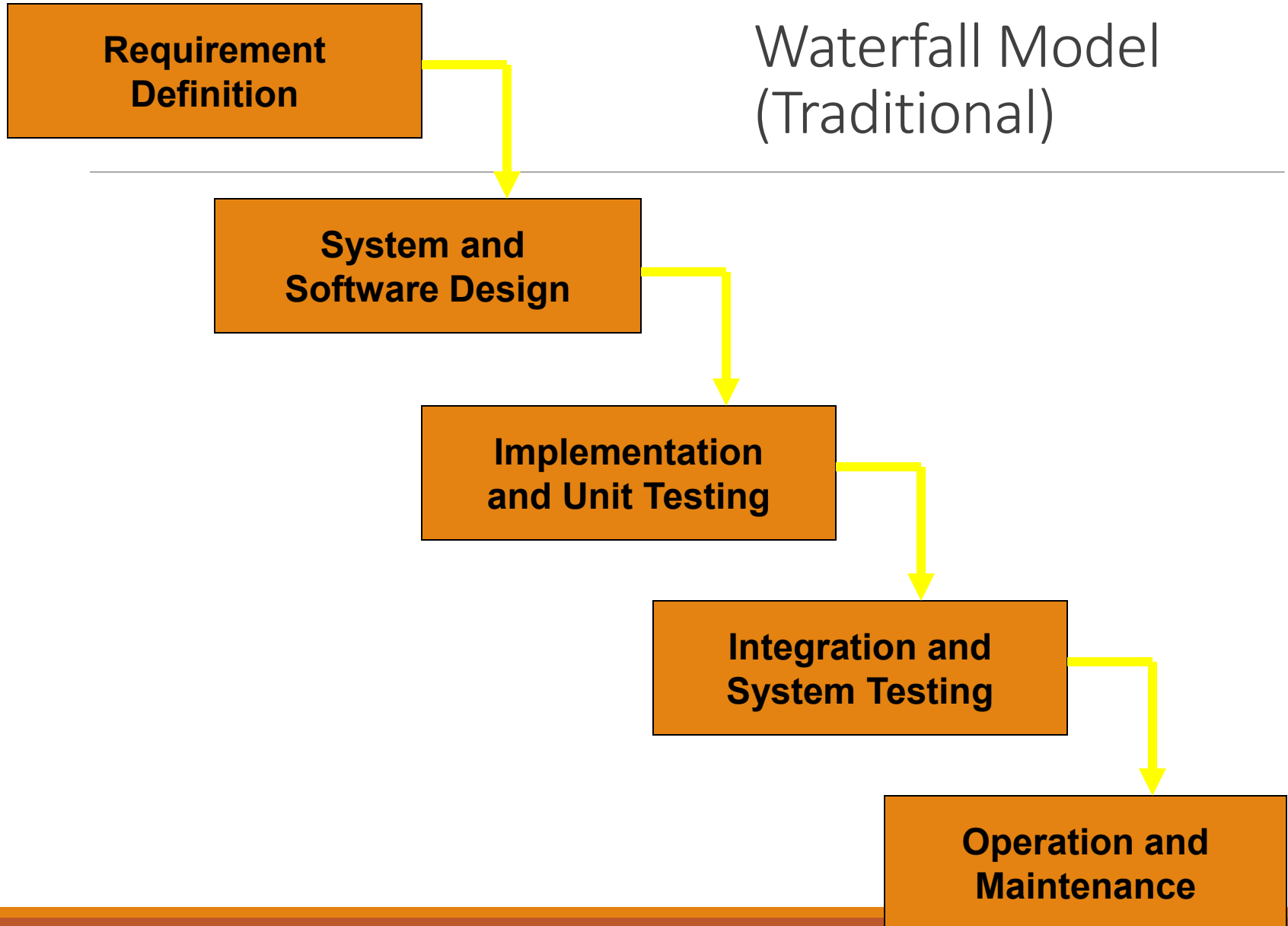
Build and Fix Model



Build and Fix Model

- It is unfortunate that many products are developed using what is known as the build-and-fix model.
- The product is constructed without specification or any attempt at design.
- The developers simply build a product that is reworked as many times as necessary to satisfy the client.
- May work for small projects but is totally unsatisfactory for products of any reasonable size.
- The cost of build-and fix is actually far greater than the cost of properly specified and carefully designed product.
- Maintenance of the product can be extremely in the absence of any documentation.

Waterfall Model (Traditional)



Waterfall Model (1)

The first published model of the software development process was derived from other engineering processes. Because of the cascade from one phase to another, this model is known as the waterfall model. This model is also known as linear sequential model. The principal stages of the model map directly onto fundamental development activities.

It suggests a systematic, sequential approach to software development that begins at the system level and progresses through the analysis, design, coding, testing, and maintenance.

In the literature, people have identified from 5 to 8 stages of software development.

Waterfall Model (2)

The five stages above are as follows:

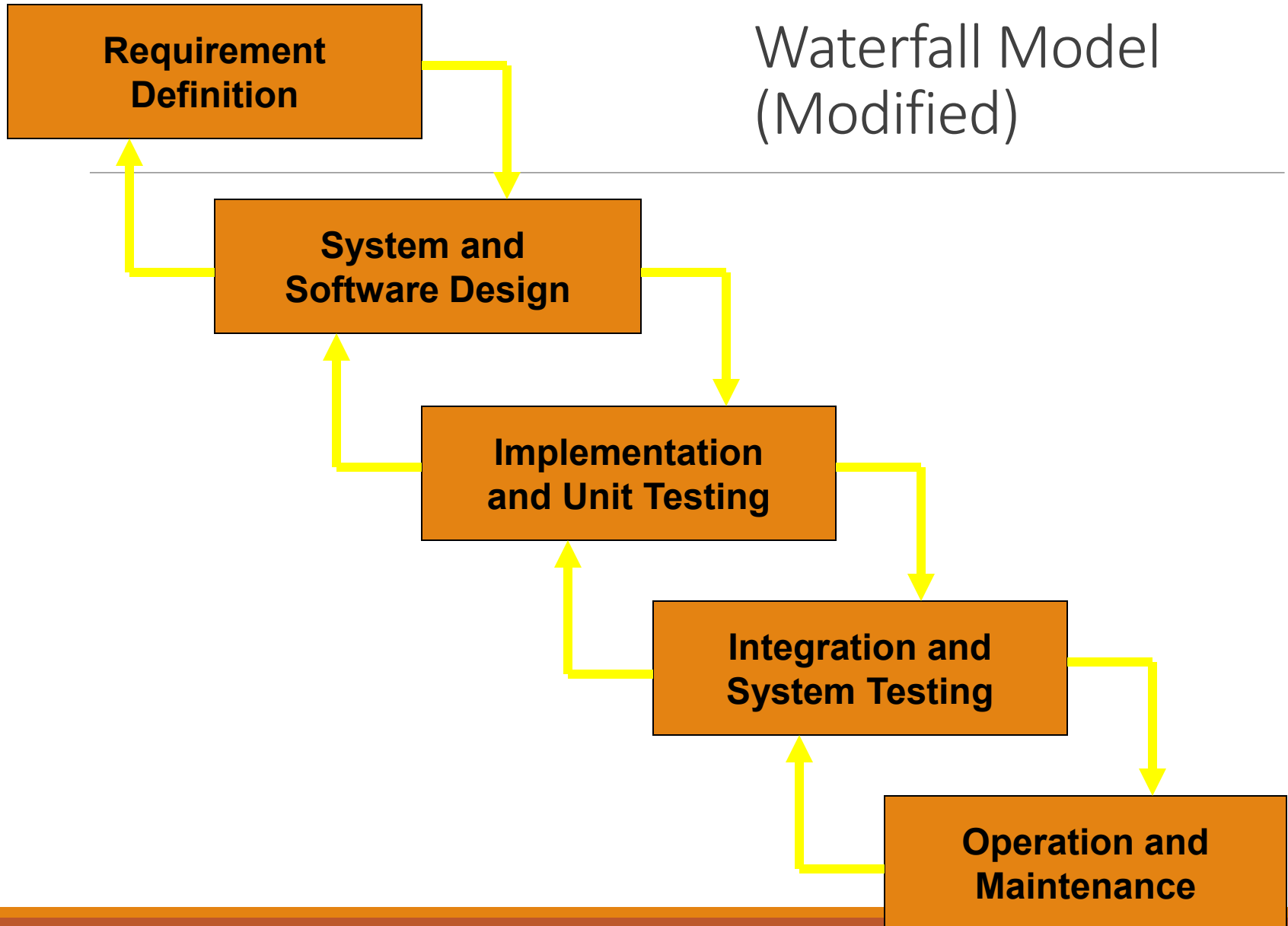
- 1. Requirement Analysis and Definition:** What - The systems services, constraints and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.
- 2. System and Software Design:** How – The system design process partitions the requirements to either hardware or software systems. It establishes an overall system architecture. Software design involves fundamental system abstractions and their relationships.
- 3. Implementation and Unit Testing:** - How – During this stage the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specifications.

Waterfall Model (3)

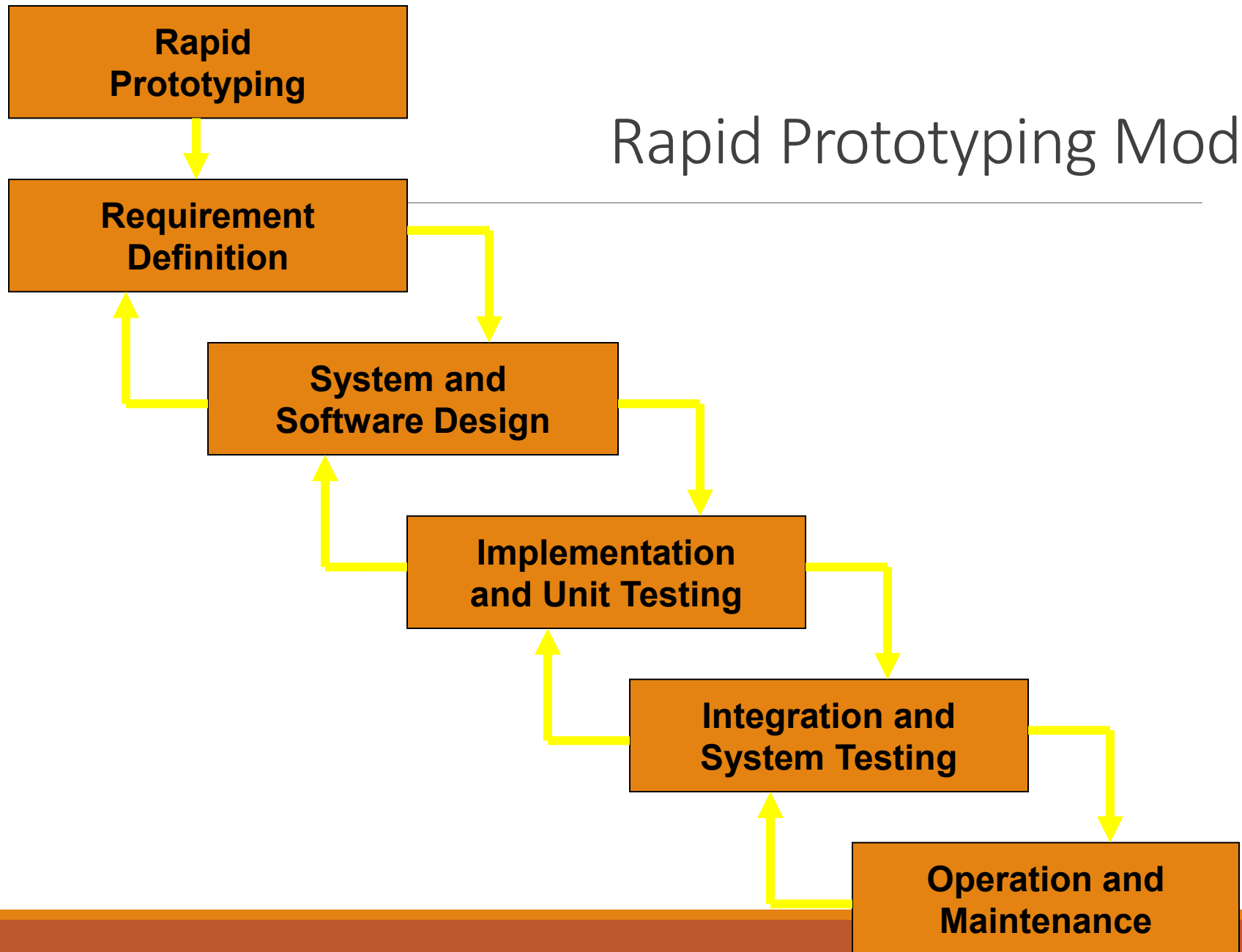
- 1. Integration and system testing:** The individual program unit or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.
- 2. Operation and Maintenance:** Normally this is the longest phase of the software life cycle. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life-cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

In principle, the result of each phase is one or more documents which are approved. No phase is complete until the documentation for that phase has been completed and products of that phase have been approved. The following phase should not start until the previous phase has finished.

Waterfall Model (Modified)



Rapid Prototyping Model



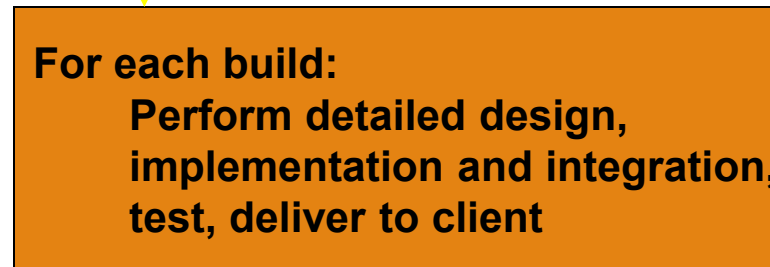
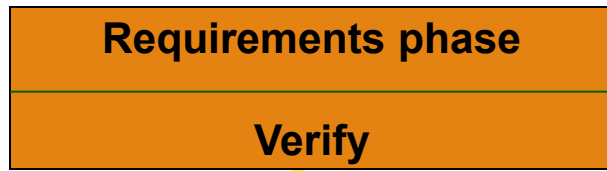
Rapid Prototyping Model

Real projects rarely follow the sequential flow that the model proposes.

In general these phases overlap and feed information to each other. Hence there should be an element of iteration and feedback.

A mistake caught any stage should be referred back to the source and all the subsequent stages need to be revisited and corresponding documents should be updated accordingly.

Because of the costs of producing and approving documents, iterations are costly and require significant rework.



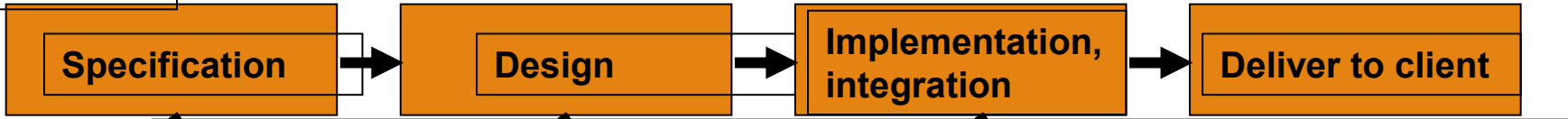
Incremental Model

 Development

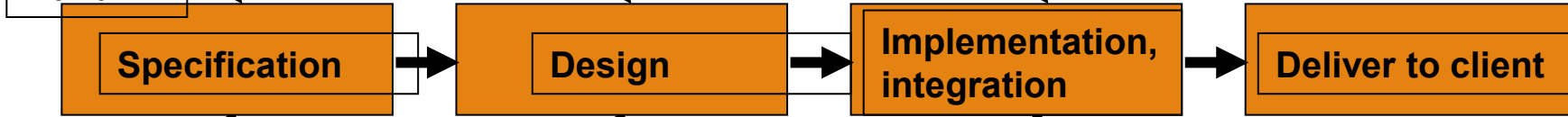
 Maintenance

Incremental Model (cont.)

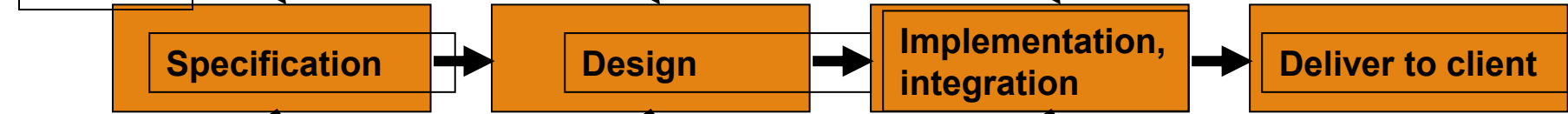
Build 1



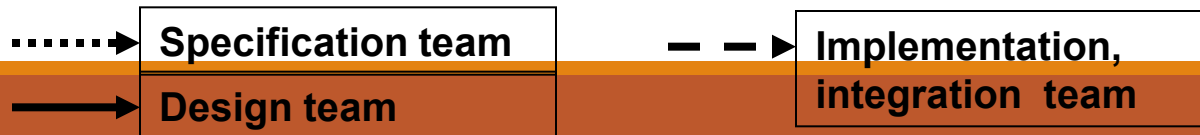
Build 2



Build 3



Build n



Advantages

- ❑ Generates working software quickly and early during the software life cycle.
- ❑ This model is more flexible – less costly to change scope and requirements.
- ❑ It is easier to test and debug during a smaller iteration.
- ❑ In this model customer can respond to each built.
- ❑ Lowers initial delivery cost.
- ❑ Easier to manage risk because risky pieces are identified and handled during each iteration.

Disadvantages

- ❑ Needs good planning and design.
- ❑ Needs a clear and complete definition of the whole system before it can be broken down and built incrementally.
- ❑ Total cost is higher than waterfall.

When to use the Incremental model:

- ❑ This model can be used when the requirements of the complete system are clearly defined and understood.
- ❑ Major requirements must be defined; however, some details can evolve with time.
- ❑ There is a need to get a product to the market early.
- ❑ A new technology is being used
- ❑ Resources with needed skill set are not available
- ❑ There are some high risk features and goals.

Risk and Risk Analysis

Risk: An uncertain event that may happen in the future and could cause negative impact.

In software engineering, a risk is a potential problem or uncertain event that may negatively affect a software project's objectives (such as cost, schedule, quality, or performance) if it occurs.

Risk analysis is the process of identifying, assessing, and managing potential problems that could negatively affect a software project.

The aim is to anticipate issues before they occur and plan mitigation strategies to reduce their impact.

Risk Analysis (Types of Risks)

Project Risks – threats to project schedule, budget, resources, or people.

Example: Key developer leaving mid-project.

Technical Risks – issues with technology, tools, or implementation.

Example: New framework not scaling under load.

Business Risks – risks that affect product acceptance or success.

Example: Customer requirements changing after release.

Operational Risks – risks related to environment, infrastructure, or deployment.

Example: Server downtime or data center failure.

Risk Analysis (Steps)

Risk Identification: Brainstorm potential risks using past projects, checklists, and expert input.

Risk Assessment (Analysis): Estimate likelihood (probability) of risk occurrence.

Estimate impact (consequence) if the risk happens.

Combine them into a risk exposure measure ($\text{Risk} = \text{Probability} \times \text{Impact}$).

Risk Prioritization: Rank risks by severity to focus on critical ones first.

Often visualized using a Risk Matrix (Low/Medium/High probability vs impact).

Risk Mitigation (Planning): Define strategies such as:

Avoidance (change plan to eliminate risk).

Mitigation (reduce probability or impact).

Transfer (shift risk to third party, e.g., insurance).

Acceptance (acknowledge and prepare for recovery).

Risk Analysis (Steps)

Risk Monitoring & Control: Continuously track risks throughout the project.

Update plans when risks evolve.

Example

Risk: Delivery deadline may slip due to underestimated complexity.

Probability: High

Impact: Severe (missed client deadline)

Mitigation: Add buffer time, hire an extra developer, or reduce non-critical features.

Risk analysis in software engineering is about foreseeing potential problems, evaluating their severity, and planning actions to handle them so projects stay on track.

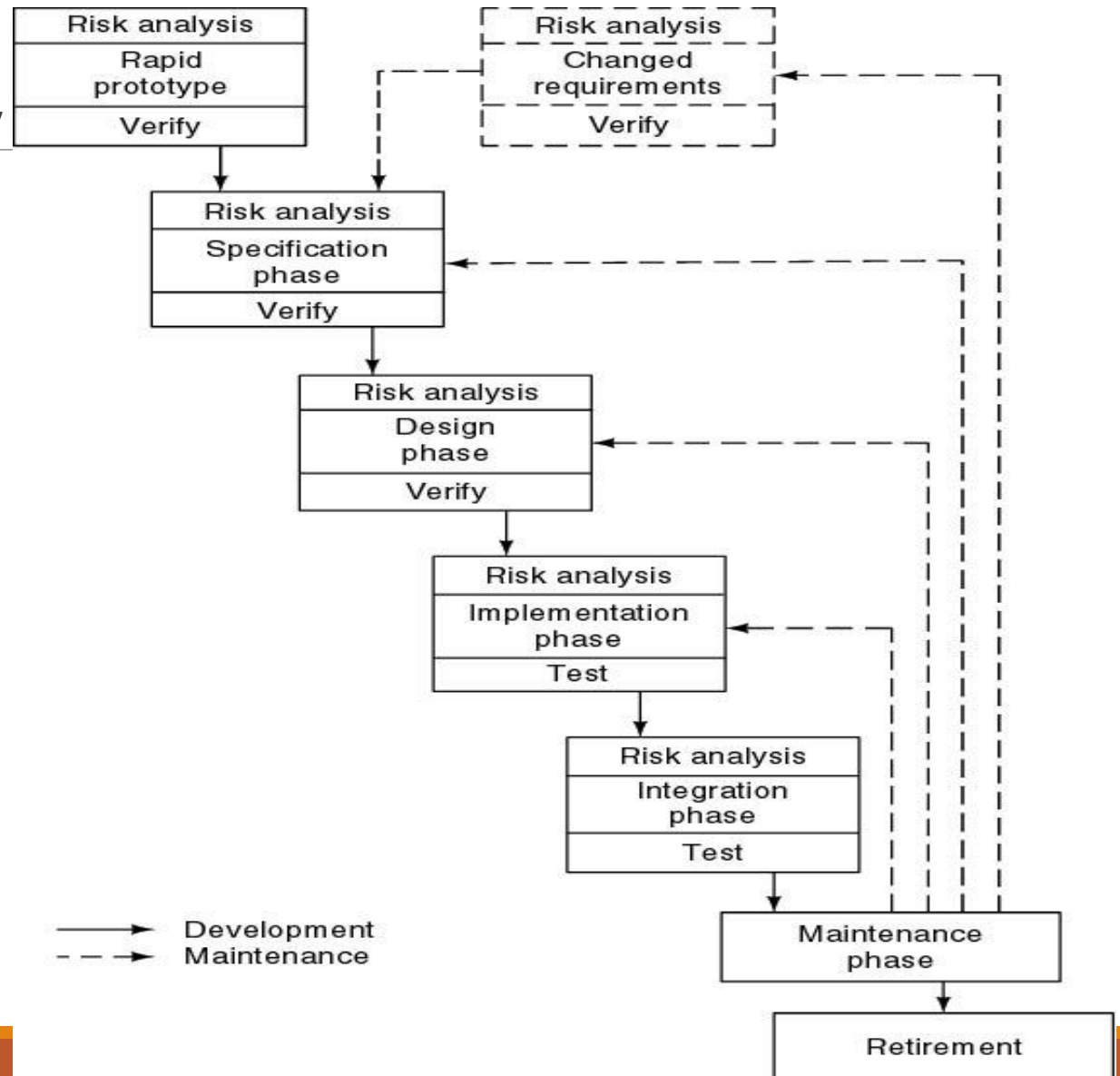
Spiral Model

Precede each phase by

- Alternatives
- Risk analysis

Follow each phase by

- Evaluation
- Planning of next phase



Spiral Model – Overview

- Proposed by Barry Boehm (1986)
- Combines iterative development with systematic risk analysis
- Project progresses through repeated cycles (spirals)
- Each spiral has 4 phases: Planning, Risk Analysis, Engineering, Evaluation

Key Characteristics

- Risk-driven approach to software development
- Each iteration delivers a more complete system
- Suitable for large, complex, and high-risk projects
- Customer feedback is emphasized at each stage

Advantages of Spiral Model

- Strong focus on risk management
- Early detection of errors and issues
- Encourages customer involvement throughout
- Accommodates changing requirements
- Well-suited for large and mission-critical systems

Disadvantages of Spiral Model

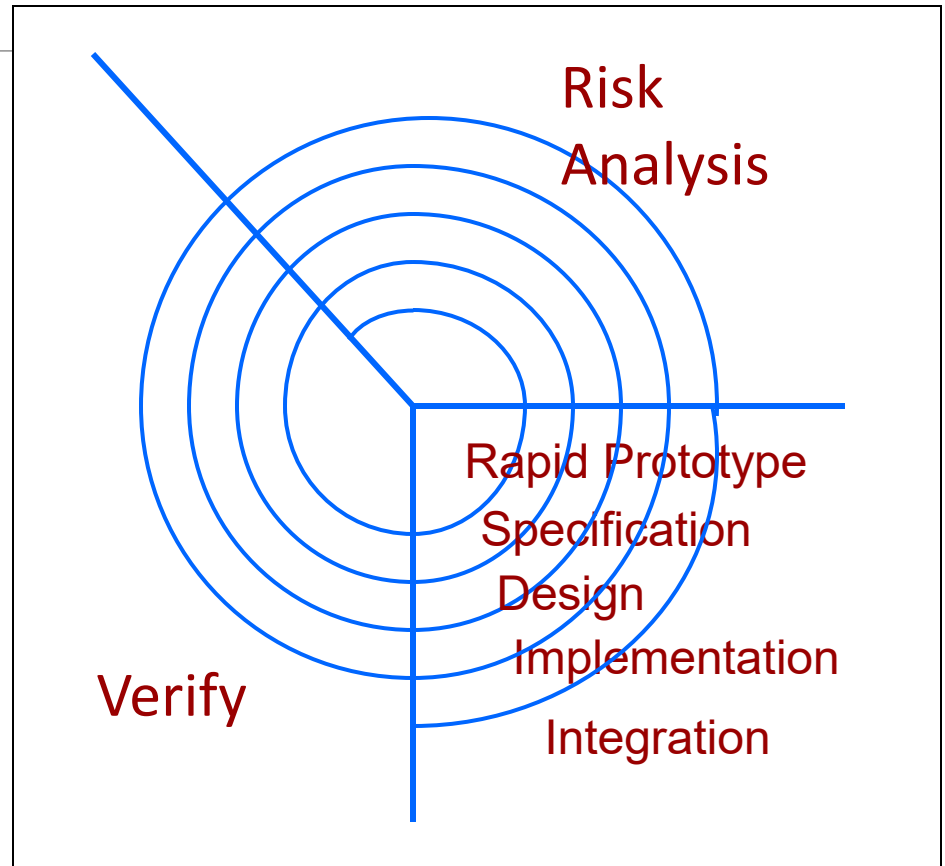
- Expensive due to repeated risk analysis and iterations
- Requires expertise in risk assessment
- Not suitable for small or low-risk projects
- Time-consuming and complex to manage
- Heavily dependent on quality of risk analysis

Examples of Spiral Model Use

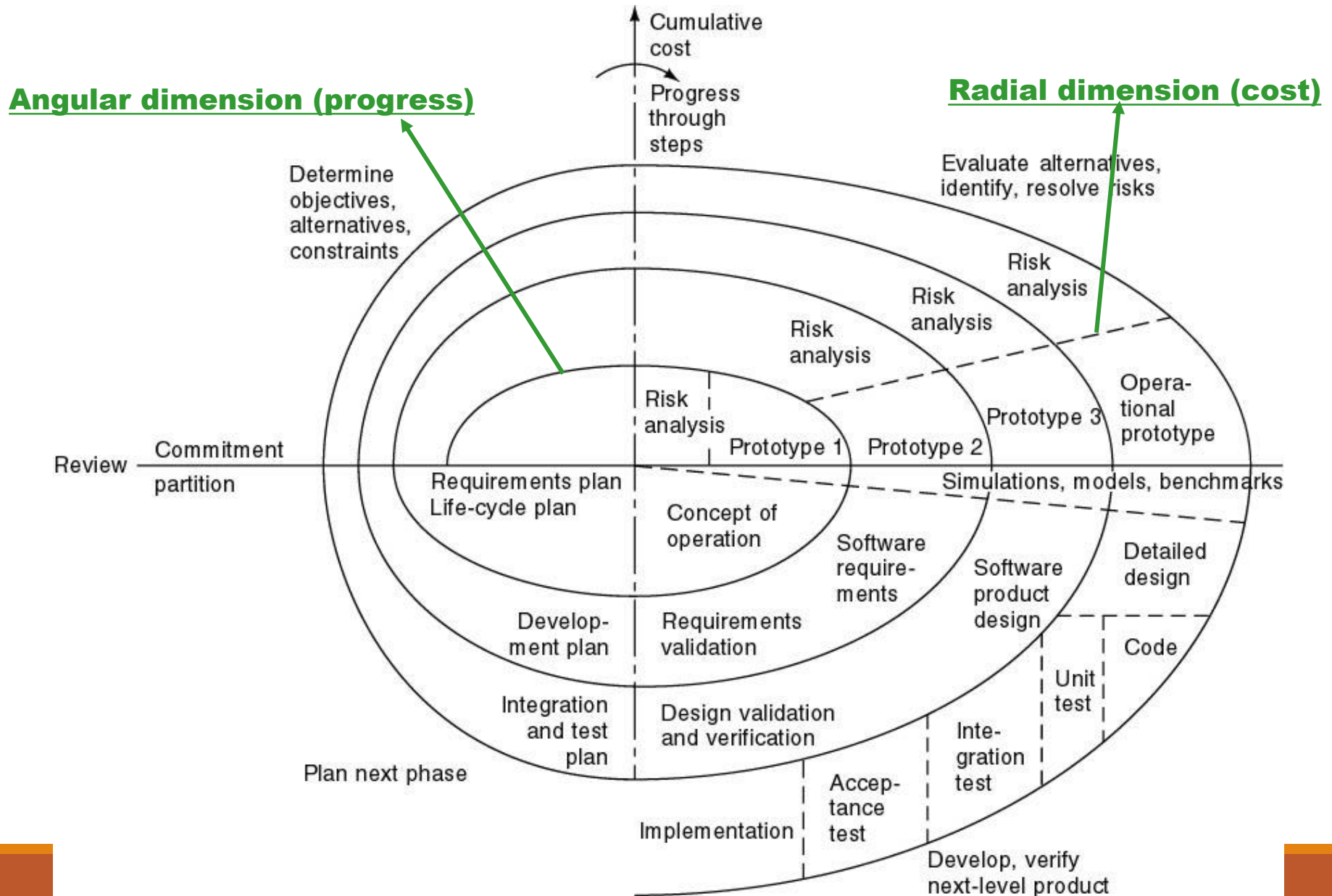
- Developing banking software where security risks must be carefully managed
- Creating aerospace systems (e.g., NASA projects) requiring reliability and risk analysis
- Designing large enterprise ERPs where requirements evolve gradually
- Game development where continuous feedback and prototyping are important

Simplified Spiral Model

If risks cannot be resolved, project is immediately terminated



Full Spiral Model



Extreme Programming (XP) – Overview

- Introduced by Kent Beck in the 1990s.
- An agile software development methodology.
- Focuses on customer satisfaction, flexibility, and teamwork.
- Emphasizes short development cycles, frequent releases, and continuous feedback.
- Encourages simple design, coding standards, and close customer-developer collaboration.

Key Practices in XP

- Pair Programming – two developers work together on the same code.
- Continuous Integration – code integrated and tested frequently.
- Small Releases – frequent delivery of working software.
- Refactoring – continuous improvement of code structure.
- Collective Code Ownership – anyone can improve any part of the code.
- 40-Hour Work Week – avoid burnout by maintaining sustainable pace.
- On-Site Customer – customer representative is always available for feedback.

Advantages of XP

- Ensures high customer satisfaction through continuous involvement.
- Early delivery of working software.
- Improves code quality through refactoring, and pair programming.
- Highly adaptable to changing requirements.
- Promotes team collaboration and communication.
- Reduces risks by catching defects early.

Disadvantages of XP

- High customer involvement required (may not always be feasible).
- Difficult to manage in large, distributed teams.
- Can be costly due to pair programming and frequent testing.
- Lacks emphasis on formal documentation.
- May not work well for projects with fixed scope and deadlines.
- Requires developers with strong communication and technical skills.

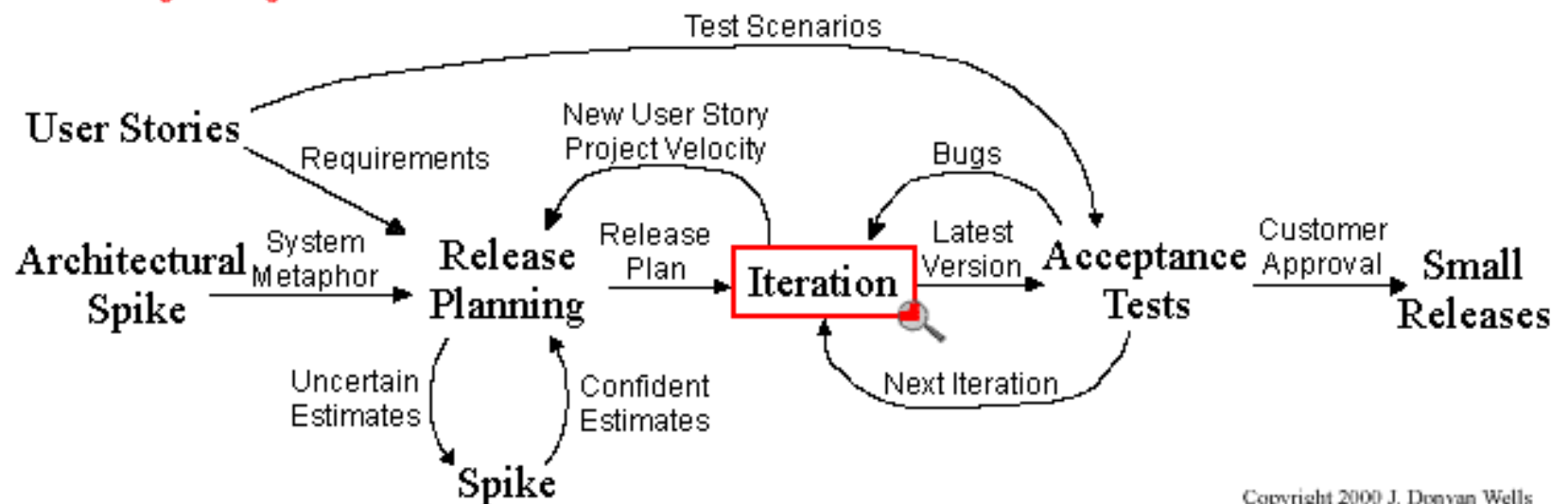
Examples of XP Use

- Startup product development – frequent changes, evolving needs.
- Web & mobile apps requiring quick releases.
- Game development with continuous testing and feedback.
- Projects with unclear or dynamic requirements.

Extreme Programming



Extreme Programming Project



Comparison of Process Models

Process Model	Strengths	Weaknesses
Build-and-fix model	• Fine for short programs that not require maintenance	• Totally unsatisfactory for nontrivial programs
Waterfall model	• Disciplined approach • Document driven	• Delivered product may not meet client's needs
Rapid prototyping model	• Ensures that delivered product meets client's needs	• Not yet proven
Incremental model	• Maximizes early return on investment • Promotes maintainability	• Requires open architecture • May degenerate into build-and-fix
Extreme programming	• Maximizes early return on investment • Fine when requirements are vague	• Small projects, small teams • Lack of design documentation • Has not yet been widely used
Synchronize-and-stabilize model	• Components always work together • Early insights into operation of product	• Has not been widely used other than at Microsoft
Spiral model	• "How much to test ?" in terms of risk • Maintenance is another cycle	• Only for large-scale, in-house products
Object-oriented models	• Iteration within phases • Parallelism between phases	• May degenerate into CABTAB

Documentation Phase?

There is NO documentation phase

Every phase must be fully documented before starting the next phase

- Postponed documentation may never be completed
- The responsible individual may leave
- The product is constantly changing—we need the documentation to do this
- The design (for example) will be modified during development, but the original designers may not be available to document it

Documents

<i>Requirement Definition</i>	• Rapid prototype, or • Requirements document	• Rapid prototype • Reviews
<i>Functional Specification</i>	• Specification document (specifications) • Software Product Management Plan	• Traceability • FS Review • Check the SPMP
<i>Design</i>	• Architectural Design • Detailed Design	• Traceability • Review
<i>Coding</i>	• Source code • Test cases	• Traceability • Review • Testing
<i>Integration</i>	• Source code • Test cases	• Integration testing • Acceptance testing
<i>Maintenance</i>	• Change record • Regression test cases	• Regression testing

Conclusions

Different life-cycle models

Each with own strengths

Each with own weaknesses

Criteria for deciding on a model include

- The organization
- Its management
- Skills of the employees
- The nature of the product

Best suggestion

- “Mix-and-match” life-cycle model